

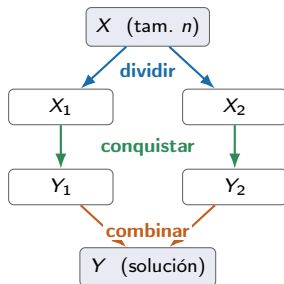
Divide & Conquer

Técnicas de Diseño de Algoritmos

Departamento de Computación, FCEyN, UBA

2 de septiembre de 2026

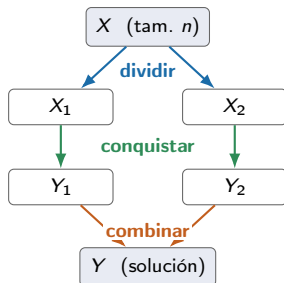
Divide & Conquer



DC(X)

```
if  $X$  es chico then
  Retornar solución ad hoc de  $X$ 
else
  Descomponer  $X$  en  $X_1, \dots, X_k$ 
  for  $i \in [1..k]$  do
     $Y_i = \text{DC}(X_i)$ 
  Combinar los  $Y_i$  en una solución de  $X$ 
```

Divide & Conquer

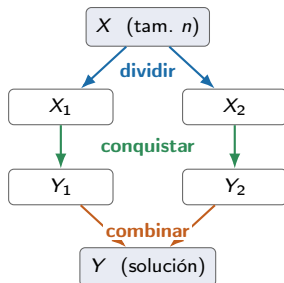


DC(X)

```
if  $X$  es chico then  
  Retornar solución ad hoc de  $X$   
else  
  Descomponer  $X$  en  $X_1, \dots, X_k$   
  for  $i \in [1..k]$  do  
     $Y_i = \text{DC}(X_i)$   
  Combinar los  $Y_i$  en una solución de  $X$ 
```

- **Dividir:** k subproblemas del mismo tipo, más chicos.

Divide & Conquer

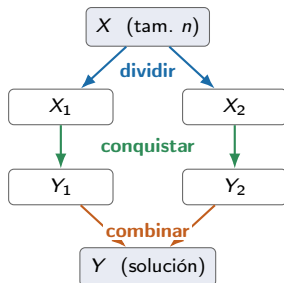


DC(X)

```
if  $X$  es chico then
  Retornar solución ad hoc de  $X$ 
else
  Descomponer  $X$  en  $X_1, \dots, X_k$ 
  for  $i \in [1..k]$  do
     $Y_i = DC(X_i)$ 
  Combinar los  $Y_i$  en una solución de  $X$ 
```

- **Dividir**: k subproblemas del mismo tipo, más chicos.
- **Conquistar**: resolverlos (recursión o caso base).

Divide & Conquer



DC(X)

```
if  $X$  es chico then  
  Retornar solución ad hoc de  $X$   
else  
  Descomponer  $X$  en  $X_1, \dots, X_k$   
  for  $i \in [1..k]$  do  
     $Y_i = \text{DC}(X_i)$   
  Combinar los  $Y_i$  en una solución de  $X$ 
```

- **Dividir**: k subproblemas del mismo tipo, más chicos.
- **Conquistar**: resolverlos (recursión o caso base).
- **Combinar**: armar con eso la solución original.

La complejidad de un algoritmo D&C

Como el algoritmo se llama a sí mismo, su costo queda escrito en función de sí mismo:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq k \\ a T(\frac{n}{b}) + f(n) & \text{si } n > k \end{cases}$$

- k : tamaño del caso base (en general, 1).
- a : cuántos subproblemas resolvemos.
- b : en cuántas partes dividimos; cada subproblema mide n/b .
- $f(n)$: lo que cuesta cada llamado *además* de la recursión, o sea dividir + combinar.

La complejidad de un algoritmo D&C

Como el algoritmo se llama a sí mismo, su costo queda escrito en función de sí mismo:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq k \\ a T\left(\frac{n}{b}\right) + f(n) & \text{si } n > k \end{cases}$$

- k : tamaño del caso base (en general, 1).
- a : cuántos subproblemas resolvemos.
- b : en cuántas partes dividimos; cada subproblema mide n/b .
- $f(n)$: lo que cuesta cada llamado *además* de la recursión, o sea dividir + combinar.

Ojo: a y b no tienen por qué ser iguales.

¿Cómo calculamos la complejidad?

Opción 1: árbol de recursión

Dibujar las llamadas, ver cuánto cuesta cada nivel y sumar.

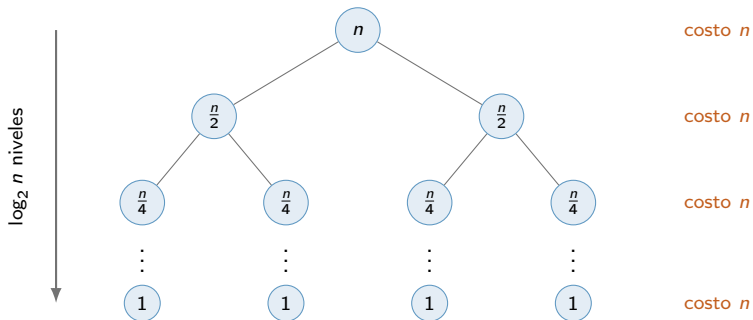
Opción 2: método de sustitución

Adivinar la cota y probarla por inducción.

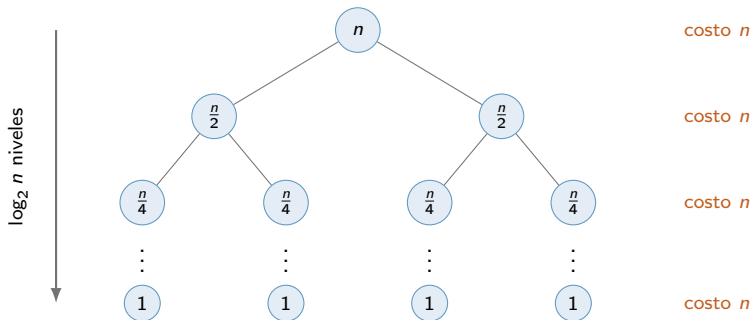
Opción 3: Teorema Maestro

Si $T(n)$ tiene la forma justa, leemos la respuesta de una tabla.

Árbol de recursión



Árbol de recursión



Cada nivel cuesta n y hay $\log_2 n$ niveles $\Rightarrow T(n) = \Theta(n \log n)$

Teorema Maestro

Si $T(n) = a T\left(\frac{n}{b}\right) + f(n)$ con $a \geq 1$ y $b > 1$, llamamos $q = \log_b a$ y comparamos $f(n)$ contra n^q :

Caso	Hipótesis	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$ con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y, eventualmente, $a f(n/b) \leq c f(n)$ para algún $0 < c < 1$	$T(n) = \Theta(f(n))$

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

¿Cómo vamos a resolver cada ejercicio?

Por lo general vamos a seguir estos cinco pasos:

1 · Análisis

entender el problema y buscar la propiedad que lo hace fácil

2 · Algoritmo

la idea, en castellano, antes de escribir código

3 · Etapas D&C

¿qué es dividir, qué es conquistar, qué es combinar?

4 · Correctitud

¿por qué lo que descartamos no tenía la respuesta?

5 · Complejidad

plantear $T(n)$ y resolverla

Ejercicio 9 (*MáximoMontaña*)

Problema

Un arreglo es *montaña* si es una secuencia estrictamente creciente seguida de una estrictamente decreciente.

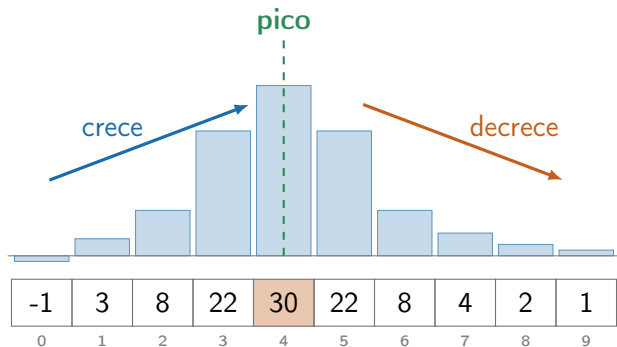
Dado un arreglo montaña de longitud n , encontrar su máximo en $O(\log n)$.

-1	3	8	22	30	22	8	4	2	1
0	1	2	3	4	5	6	7	8	9

El máximo es **30**, en la posición **4**.

Ejercicio 9

1 · Análisis

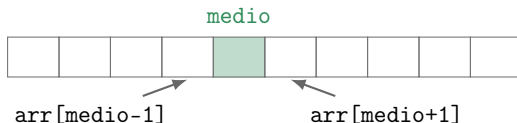


Ejercicio 9

2 · Algoritmo

La intuición

El pico es el **único** lugar donde el arreglo deja de crecer: comparando $\text{arr}[\text{medio}]$ con sus vecinos ya sé de qué lado está.

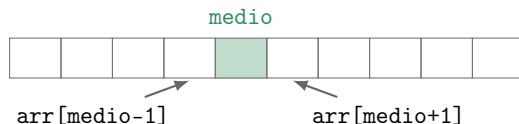


Ejercicio 9

2 · Algoritmo

La intuición

El pico es el **único** lugar donde el arreglo deja de crecer: comparando $\text{arr}[\text{medio}]$ con sus vecinos ya sé de qué lado está.



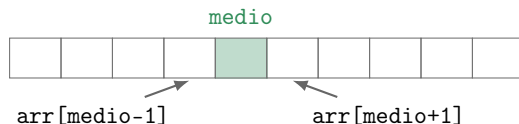
- $\text{arr}[\text{medio}] < \text{arr}[\text{medio}-1]$: estoy bajando, el pico quedó **a la izquierda**.

Ejercicio 9

2 · Algoritmo

La intuición

El pico es el **único** lugar donde el arreglo deja de crecer: comparando $\text{arr}[\text{medio}]$ con sus vecinos ya sé de qué lado está.



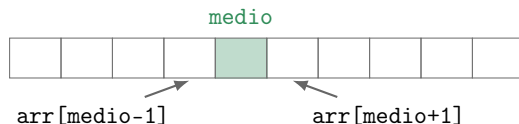
- $\text{arr}[\text{medio}] < \text{arr}[\text{medio}-1]$: estoy bajando, el pico quedó **a la izquierda**.
- $\text{arr}[\text{medio}] < \text{arr}[\text{medio}+1]$: estoy subiendo, el pico está **a la derecha**.

Ejercicio 9

2 · Algoritmo

La intuición

El pico es el **único** lugar donde el arreglo deja de crecer: comparando $\text{arr}[\text{medio}]$ con sus vecinos ya sé de qué lado está.



- $\text{arr}[\text{medio}] < \text{arr}[\text{medio}-1]$: estoy bajando, el pico quedó **a la izquierda**.
- $\text{arr}[\text{medio}] < \text{arr}[\text{medio}+1]$: estoy subiendo, el pico está **a la derecha**.
- Ninguna de las dos: **arr[medio] es el pico**.

Ejercicio 9

3 · Etapas D&C

```
maximo_montana(arr, izq, der)
```

```
if izq == der: return izq
medio = (izq + der) // 2
```

```
if medio > 0 and arr[medio] < arr[medio - 1]:
    return maximo_montana(arr, izq, medio - 1)
elif medio < len(arr) - 1 and arr[medio] < arr[medio + 1]:
    return maximo_montana(arr, medio + 1, der)
else:
    return medio
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 9

3 · Etapas D&C

```
maximo_montana(arr, izq, der)
```

```
if izq == der: return izq
medio = (izq + der) // 2
```

```
if medio > 0 and arr[medio] < arr[medio - 1]:
    return maximo_montana(arr, izq, medio - 1)
elif medio < len(arr) - 1 and arr[medio] < arr[medio + 1]:
    return maximo_montana(arr, medio + 1, der)
else:
    return medio
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 9

3 · Etapas D&C

```
maximo_montana(arr, izq, der)
```

```
if izq == der: return izq
medio = (izq + der) // 2
```

```
if medio > 0 and arr[medio] < arr[medio - 1]:
    return maximo_montana(arr, izq, medio - 1)
elif medio < len(arr) - 1 and arr[medio] < arr[medio + 1]:
    return maximo_montana(arr, medio + 1, der)
else:
    return medio
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 9

3 · Etapas D&C

```
maximo_montana(arr, izq, der)
```

```
if izq == der: return izq
medio = (izq + der) // 2

if medio > 0 and arr[medio] < arr[medio - 1]:
    return maximo_montana(arr, izq, medio - 1)
elif medio < len(arr) - 1 and arr[medio] < arr[medio + 1]:
    return maximo_montana(arr, medio + 1, der)
else:
    return medio
```

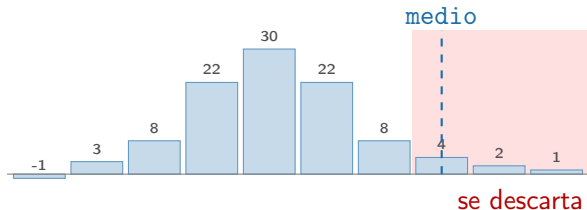
■ Dividir ■ Conquistar ■ Combinar

Ojo: se ejecuta **una sola** llamada recursiva, y *combinar* no hace nada: devolvemos el resultado tal cual.

Ejercicio 9

4 · Correctitud

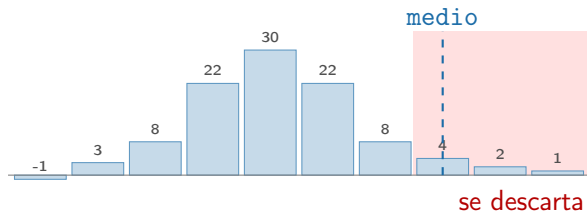
Invariante: el pico siempre está dentro de $[izq, der]$.



Ejercicio 9

4 · Correctitud

Invariante: el pico siempre está dentro de $[izq, der]$.



Si $arr[medio] < arr[medio-1]$, desde $medio$ en adelante el arreglo **sólo decrece**: ningún elemento de la derecha puede ser el máximo.

Ejercicio 9

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 1 \end{cases}$$

Ejercicio 9

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ T(\frac{n}{2}) + \Theta(1) & \text{si } n > 1 \end{cases}$$

- $a = 1$ (una sola llamada), $b = 2$, $f(n) = \Theta(1)$
- $q = \log_b a = \log_2 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Ejercicio 9

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 1 \end{cases}$$

- $a = 1$ (una sola llamada), $b = 2$, $f(n) = \Theta(1)$
- $q = \log_b a = \log_2 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

Ejercicio 9

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 1 \end{cases}$$

- $a = 1$ (una sola llamada), $b = 2$, $f(n) = \Theta(1)$
- $q = \log_b a = \log_2 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

$$T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

Ejercicio 9 - Ejemplo de ejecución

(1)

1	5	9	7	3	2	1
---	---	---	---	---	---	---

0	1	2	3	4	5	6
---	---	---	---	---	---	---

 $7 < 9 \Rightarrow$ izquierda

(2)

1	5	9	7	3	2	1
---	---	---	---	---	---	---

0	1	2	3	4	5	6
---	---	---	---	---	---	---

 $5 < 9 \Rightarrow$ derecha

(3)

1	5	9	7	3	2	1
---	---	---	---	---	---	---

0	1	2	3	4	5	6
---	---	---	---	---	---	---

 máximo en la posición 2

En gris, la ventana $[izq, der]$; en verde, medio.

Ejercicio 9 - Para comparar

Enfoque	Complejidad	Idea
Fuerza bruta	$O(n)$	Recorrer todo el arreglo
Divide & Conquer	$O(\log n)$	Descartar media montaña por paso

Ejercicio 9 - Para comparar

Enfoque	Complejidad	Idea
Fuerza bruta	$O(n)$	Recorrer todo el arreglo
Divide & Conquer	$O(\log n)$	Descartar media montaña por paso

¿Qué hizo la diferencia?

Aprovechar que el arreglo es **unimodal**.

Búsqueda Lineal Modificada

Problema

Dado un arreglo A y un elemento $elem$, determinar si $elem$ está en A .

Pregunta clave

¿Todo algoritmo recursivo que *divide* el problema es *divide & conquer*?

Búsqueda Lineal Modificada

Problema

Dado un arreglo A y un elemento $elem$, determinar si $elem$ está en A .

Pregunta clave

¿Todo algoritmo recursivo que *divide* el problema es *divide & conquer*?

No.

Primera versión

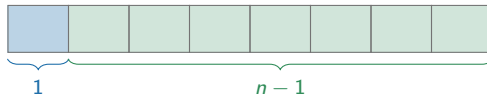
```
BúsquedaLinealModificada(A, elem)
```

```
if |A| == 0: return false  
if |A| == 1: return elem == A[0]  
return BúsquedaLinealModificada(subarray(A, 0, 1), elem)  
    or BúsquedaLinealModificada(subarray(A, 1, |A|), elem)
```

Primera versión

BúsquedaLinealModificada(A, elem)

```
if |A| == 0: return false
if |A| == 1: return elem == A[0]
return BúsquedaLinealModificada(subarray(A, 0, 1), elem)
       or BúsquedaLinealModificada(subarray(A, 1, |A|), elem)
```



¿Por qué NO es Divide & Conquer?

D&C pide...

- División **balanceada**.
- Subproblemas **una fracción** del original.

Acá pasa...

- $n \rightarrow 1 + (n - 1)$.
- Achicamos **un solo elemento**.

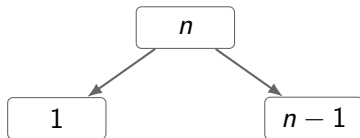
¿Por qué NO es Divide & Conquer?

D&C pide...

- División **balanceada**.
- Subproblemas **una fracción** del original.

Acá pasa...

- $n \rightarrow 1 + (n - 1)$.
- Achicamos **un solo elemento**.



Complejidad de la primera versión

$$T(n) = T(1) + T(n-1) + O(1) = T(n-1) + O(1)$$

Complejidad de la primera versión

$$T(n) = T(1) + T(n-1) + O(1) = T(n-1) + O(1)$$

$$\begin{aligned} T(n) &= T(n-1) + O(1) = T(n-2) + 2 \cdot O(1) = \dots \\ &= T(1) + (n-1) \cdot O(1) = O(n) \end{aligned}$$

Complejidad de la primera versión

$$T(n) = T(1) + T(n-1) + O(1) = T(n-1) + O(1)$$

$$\begin{aligned} T(n) &= T(n-1) + O(1) = T(n-2) + 2 \cdot O(1) = \dots \\ &= T(1) + (n-1) \cdot O(1) = O(n) \end{aligned}$$

Es una recursión lineal disfrazada.

Segunda versión

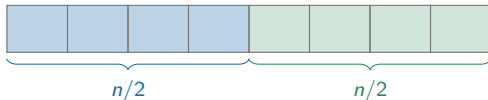
BúsquedaLinealModV2(A, elem)

```
if |A| == 0: return false
if |A| == 1: return elem == A[0]
return BúsquedaLinealModV2(subarray(A, 0, |A|/2), elem)
       or BúsquedaLinealModV2(subarray(A, |A|/2, |A|), elem)
```

Segunda versión

BúsquedaLinealModV2(A, elem)

```
if |A| == 0: return false
if |A| == 1: return elem == A[0]
return BúsquedaLinealModV2(subarray(A, 0, |A|/2), elem)
       or BúsquedaLinealModV2(subarray(A, |A|/2, |A|), elem)
```



¿Por qué SÍ es Divide & Conquer?

Ahora sí

- $n \rightarrow n/2 + n/2$.
- Achicamos a la mitad en cada paso.

Pero...

- Igual hay que mirar todos los elementos.
- No mejora la complejidad.

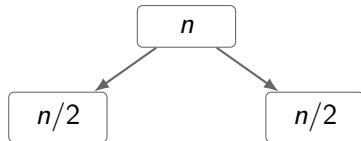
¿Por qué SÍ es Divide & Conquer?

Ahora sí

- $n \rightarrow n/2 + n/2$.
- Achicamos a la mitad en cada paso.

Pero...

- Igual hay que mirar **todos** los elementos.
- No mejora la complejidad.



Complejidad de la segunda versión

$$T(n) = 2 T(n/2) + O(1)$$

Complejidad de la segunda versión

$$T(n) = 2 T(n/2) + O(1)$$

- $a = 2, \quad b = 2, \quad f(n) = O(1)$
- $q = \log_b a = \log_2 2 = 1, \quad \text{y } f(n) = O(n^0) = O(n^{q-1})$

Complejidad de la segunda versión

$$T(n) = 2 T(n/2) + O(1)$$

- $a = 2, \quad b = 2, \quad f(n) = O(1)$
- $q = \log_b a = \log_2 2 = 1, \quad \text{y } f(n) = O(n^0) = O(n^{q-1})$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

Complejidad de la segunda versión

$$T(n) = 2 T(n/2) + O(1)$$

- $a = 2, \quad b = 2, \quad f(n) = O(1)$
- $q = \log_b a = \log_2 2 = 1, \quad \text{y } f(n) = O(n^0) = O(n^{q-1})$

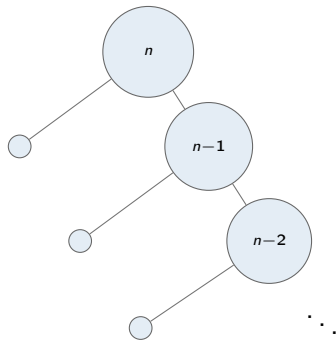
Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

$$T(n) = \Theta(n^q) = \Theta(n)$$

Es D&C de verdad, pero sigue costando $\Theta(n)$.

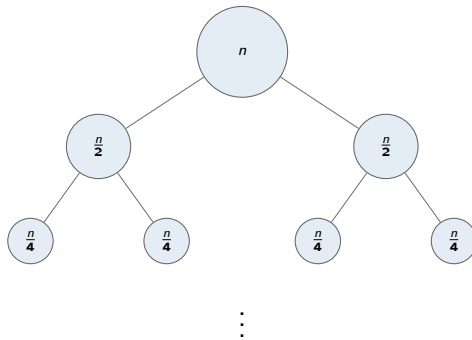
Los dos árboles de llamadas

V1: degenerado



altura $\Theta(n)$

V2: balanceado



altura $\Theta(\log n)$

El tamaño de cada nodo es proporcional al del subproblema.

Conclusión

NO es D&C

- Subproblema de tamaño $n - k$.
- División muy desbalanceada.

SÍ es D&C

- Subproblemas de tamaño n/b .
- Reducción por factor constante.

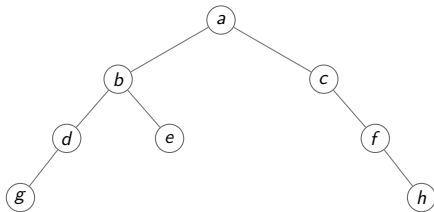
D&C necesita una división **significativa y balanceada**
... y aun así, no siempre baja la complejidad.

Ejercicio 7 (*Distancia máxima*)

Problema

Dado un árbol binario cualquiera, dar un algoritmo de divide & conquer que devuelva el **tamaño del camino más largo**.

El algoritmo **no debe hacer recorridos innecesarios** sobre el árbol.

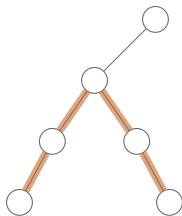


Medimos el tamaño del camino en **cantidad de aristas**.

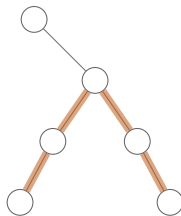
Ejercicio 7

1 · Análisis

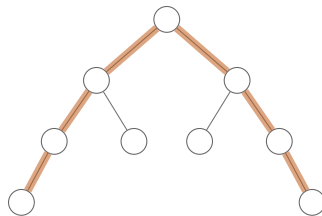
El camino más largo, ¿por dónde puede pasar?



entero en el
subárbol izquierdo



entero en el
subárbol derecho

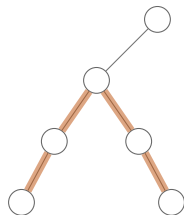


pasa
por la raíz

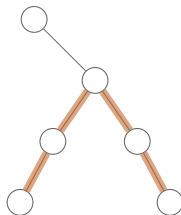
Ejercicio 7

1 · Análisis

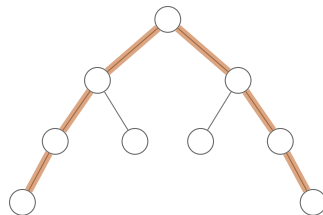
El camino más largo, ¿por dónde puede pasar?



entero en el
subárbol izquierdo



entero en el
subárbol derecho



pasa
por la raíz

La clave

Para el tercer caso no me alcanza con los caminos más largos de los subárboles: necesito sus **alturas**.

Ejercicio 7

2 · Algoritmo

Llamemos $h(T)$ a la altura y $D(T)$ al camino más largo. Entonces:

$$D(T) = \max \left(\underbrace{D(T_{izq})}_{\text{caso 1}}, \underbrace{D(T_{der})}_{\text{caso 2}}, \underbrace{h(T_{izq}) + h(T_{der}) + 2}_{\text{caso 3}} \right)$$

Ejercicio 7

2 · Algoritmo

Llamemos $h(T)$ a la altura y $D(T)$ al camino más largo. Entonces:

$$D(T) = \max \left(\underbrace{D(T_{izq})}_{\text{caso 1}}, \underbrace{D(T_{der})}_{\text{caso 2}}, \underbrace{h(T_{izq}) + h(T_{der}) + 2}_{\text{caso 3}} \right)$$

- Con $h(\emptyset) = -1$ y $D(\emptyset) = 0$.
- El $+2$ son las dos aristas que bajan de la raíz a cada subárbol.

Ejercicio 7

2 · Algoritmo

Llamemos $h(T)$ a la altura y $D(T)$ al camino más largo. Entonces:

$$D(T) = \max \left(\underbrace{D(T_{izq})}_{\text{caso 1}}, \underbrace{D(T_{der})}_{\text{caso 2}}, \underbrace{h(T_{izq}) + h(T_{der}) + 2}_{\text{caso 3}} \right)$$

- Con $h(\emptyset) = -1$ y $D(\emptyset) = 0$.
- El $+2$ son las dos aristas que bajan de la raíz a cada subárbol.

Pero además necesito $h(T) = 1 + \max(h(T_{izq}), h(T_{der}))$.

Ejercicio 7 - Sin recorridos de más

Ayuda del enunciado

Para saber el camino más largo de un árbol, posiblemente necesite conocer más que sólo los caminos más largos de sus subárboles.

Lo ingenuo

Calcular D recursivamente y, en cada nodo, llamar aparte a `altura()`.

Cada nodo se recorre muchas veces: $O(n^2)$ en el peor caso.

Lo que queremos

Que cada llamada devuelva el par (h, D) .

Un solo recorrido, cada nodo visitado una vez: $\Theta(n)$.

Ejercicio 7

3 · Etapas D&C

diametro(T) devuelve el par (altura, camino)

```
if T es vacío: return (-1, 0)
Tizq, Tder = T.izq, T.der
(hi, Di) = diametro(Tizq)
(hd, Dd) = diametro(Tder)
altura = 1 + max(hi, hd)
camino = max(Di, Dd, hi + hd + 2)
return (altura, camino)
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 7

3 · Etapas D&C

diametro(T) devuelve el par (altura, camino)

```
if T es vacío: return (-1, 0)
Tizq, Tder = T.izq, T.der
(hi, Di) = diametro(Tizq)
(hd, Dd) = diametro(Tder)
altura = 1 + max(hi, hd)
camino = max(Di, Dd, hi + hd + 2)
return (altura, camino)
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 7

3 · Etapas D&C

diametro(T) devuelve el par (altura, camino)

```
if T es vacío: return (-1, 0)
Tizq, Tder = T.izq, T.der
(hi, Di) = diametro(Tizq)
(hd, Dd) = diametro(Tder)
altura = 1 + max(hi, hd)
camino = max(Di, Dd, hi + hd + 2)
return (altura, camino)
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 7

3 · Etapas D&C

`diametro(T)` devuelve el par (altura, camino)

```
if T es vacío: return (-1, 0)
Tizq, Tder = T.izq, T.der
(hi, Di) = diametro(Tizq)
(hd, Dd) = diametro(Tder)
altura = 1 + max(hi, hd)
camino = max(Di, Dd, hi + hd + 2)
return (altura, camino)
```

■ Dividir ■ Conquistar ■ Combinar

Acá *dividir* es gratis (el árbol ya viene partido) y todo el trabajo está en *combinar*, que cuesta $\Theta(1)$.

Ejercicio 7

4 · Correctitud

Los tres casos son exhaustivos

Sea P el camino más largo y r la raíz. Como P es un camino en un árbol, o bien contiene a r , o bien no.

Ejercicio 7

4 · Correctitud

Los tres casos son exhaustivos

Sea P el camino más largo y r la raíz. Como P es un camino en un árbol, o bien contiene a r , o bien no.

- Si **no** contiene a r : P vive entero en uno de los dos subárboles $\Rightarrow$ lo devuelve esa llamada recursiva.

Ejercicio 7

4 · Correctitud

Los tres casos son exhaustivos

Sea P el camino más largo y r la raíz. Como P es un camino en un árbol, o bien contiene a r , o bien no.

- Si **no** contiene a r : P vive entero en uno de los dos subárboles $\Rightarrow$ lo devuelve esa llamada recursiva.
- Si **sí** contiene a r : P baja por izquierda y por derecha lo más posible, o sea $h(T_{izq}) + h(T_{der}) + 2$.

Ejercicio 7

4 · Correctitud

Los tres casos son exhaustivos

Sea P el camino más largo y r la raíz. Como P es un camino en un árbol, o bien contiene a r , o bien no.

- Si **no** contiene a r : P vive entero en uno de los dos subárboles $\Rightarrow$ lo devuelve esa llamada recursiva.
- Si **sí** contiene a r : P baja por izquierda y por derecha lo más posible, o sea $h(T_{izq}) + h(T_{der}) + 2$.

Como tomamos el máximo de los tres, nunca nos pasamos ni nos quedamos cortos.

Ejercicio 7

5 · Complejidad

Si el subárbol izquierdo tiene k nodos:

$$T(n) = T(k) + T(n - k - 1) + \Theta(1)$$

Ejercicio 7

5 · Complejidad

Si el subárbol izquierdo tiene k nodos:

$$T(n) = T(k) + T(n - k - 1) + \Theta(1)$$

Ojo

Acá el Teorema Maestro **no aplica**: el árbol puede estar desbalanceado, así que los subproblemas no son de tamaño n/b .

Ejercicio 7

5 · Complejidad

Si el subárbol izquierdo tiene k nodos:

$$T(n) = T(k) + T(n - k - 1) + \Theta(1)$$

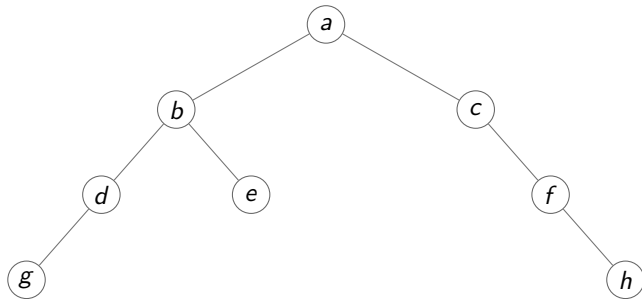
Ojo

Acá el Teorema Maestro **no aplica**: el árbol puede estar desbalanceado, así que los subproblemas no son de tamaño n/b .

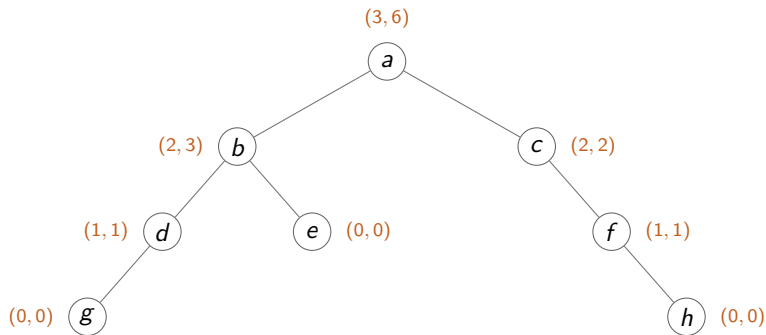
Pero cada nodo se visita **exactamente una vez** y hace $\Theta(1)$ de trabajo $\Rightarrow$

$$T(n) = \Theta(n).$$

Ejercicio 7 - Ejemplo

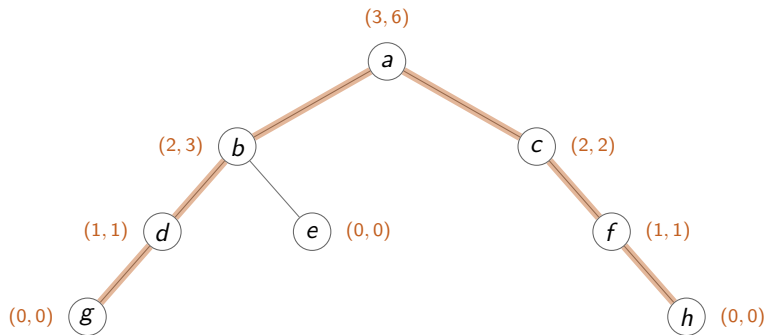


Ejercicio 7 - Ejemplo



Cada nodo devuelve su par (altura, camino).

Ejercicio 7 - Ejemplo



Cada nodo devuelve su par (altura, camino). El máximo es **6**, y sale del caso *pasa por la raíz*: $h(b) + h(c) + 2 = 2 + 2 + 2$, aunque el mejor camino de un subárbol solo era 3.

Ejercicio 13 (*Diferencia Mínima*)

Problema

Dos arreglos de n naturales: A **creciente** y B **decreciente**, sin repetidos dentro de cada uno.

Buscamos $\min_i |A[i] - B[i]|$, en $O(\log n)$.

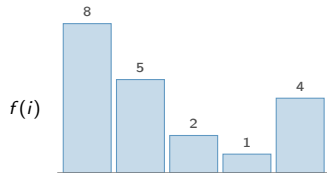
A	1	2	3	4	5	 crece
B	9	7	5	3	1	 decrece
$ A - B $	8	5	2	1	4	

Resultado: **1**.

Ejercicio 13

1 · Análisis

Le ponemos nombre a lo que queremos minimizar: $f(i) = |A[i] - B[i]|$ para cada posición i .

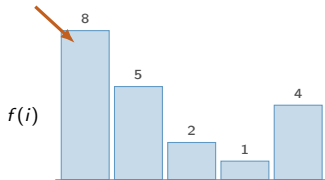


Ejercicio 13

1 · Análisis

Le ponemos nombre a lo que queremos minimizar: $f(i) = |A[i] - B[i]|$ para cada posición i .

A en su **mínimo**, B en su **máximo**
⇒ diferencia grande

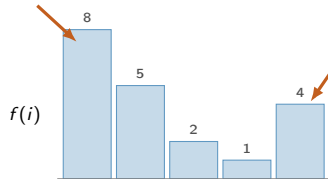


Ejercicio 13

1 · Análisis

Le ponemos nombre a lo que queremos minimizar: $f(i) = |A[i] - B[i]|$ para cada posición i .

A en su **mínimo**, B en su **máximo**
⇒ diferencia grande



A en su **máximo**, B en su **mínimo**
⇒ diferencia grande

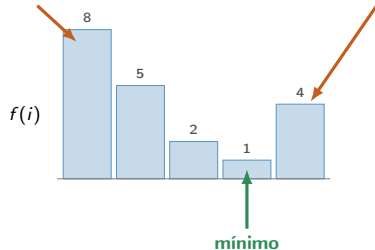
Ejercicio 13

1 · Análisis

Le ponemos nombre a lo que queremos minimizar: $f(i) = |A[i] - B[i]|$ para cada posición i .

A en su **mínimo**, B en su **máximo**
⇒ diferencia grande

A en su **máximo**, B en su **mínimo**
⇒ diferencia grande



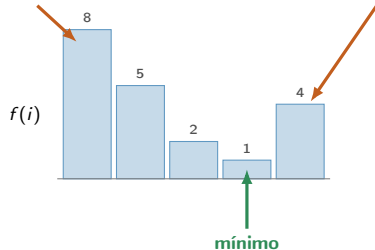
Ejercicio 13

1 · Análisis

Le ponemos nombre a lo que queremos minimizar: $f(i) = |A[i] - B[i]|$ para cada posición i .

A en su **mínimo**, B en su **máximo**
⇒ diferencia grande

A en su **máximo**, B en su **mínimo**
⇒ diferencia grande

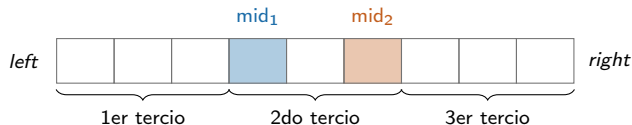


f es **unimodal**: baja y después sube (forma de V).

Ejercicio 13

2 · Algoritmo: búsqueda ternaria

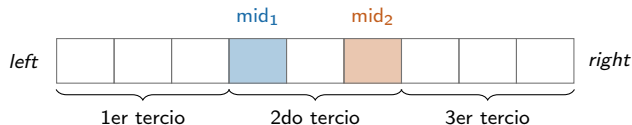
$left$, $right$, mid_1 y mid_2 son **posiciones** del arreglo. Partimos el rango $[left, right]$ en tres y evaluamos f en los dos cortes:



Ejercicio 13

2 · Algoritmo: búsqueda ternaria

$left$, $right$, mid_1 y mid_2 son **posiciones** del arreglo. Partimos el rango $[left, right]$ en tres y evaluamos f en los dos cortes:

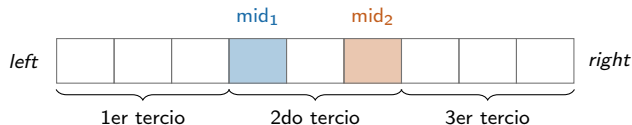


- Si $f(mid_1) > f(mid_2)$: el mínimo está a la derecha $\Rightarrow$ tiro el **primer tercio** ($left = mid_1$).

Ejercicio 13

2 · Algoritmo: búsqueda ternaria

$left$, $right$, mid_1 y mid_2 son **posiciones** del arreglo. Partimos el rango $[left, right]$ en tres y evaluamos f en los dos cortes:

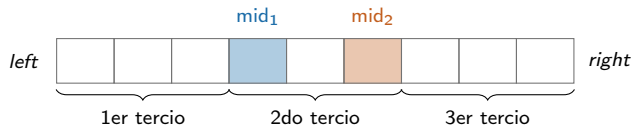


- Si $f(mid_1) > f(mid_2)$: el mínimo está a la derecha $\Rightarrow$ tiro el **primer tercio** ($left = mid_1$).
- Si $f(mid_1) \leq f(mid_2)$: el mínimo está a la izquierda $\Rightarrow$ tiro el **último tercio** ($right = mid_2$).

Ejercicio 13

2 · Algoritmo: búsqueda ternaria

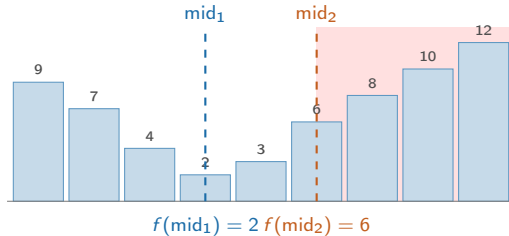
$left$, $right$, mid_1 y mid_2 son **posiciones** del arreglo. Partimos el rango $[left, right]$ en tres y evaluamos f en los dos cortes:



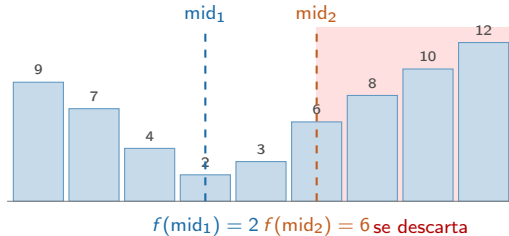
- Si $f(mid_1) > f(mid_2)$: el mínimo está a la derecha $\Rightarrow$ tiro el **primer tercio** ($left = mid_1$).
- Si $f(mid_1) \leq f(mid_2)$: el mínimo está a la izquierda $\Rightarrow$ tiro el **último tercio** ($right = mid_2$).

En cualquier caso me queda 2/3 del rango.

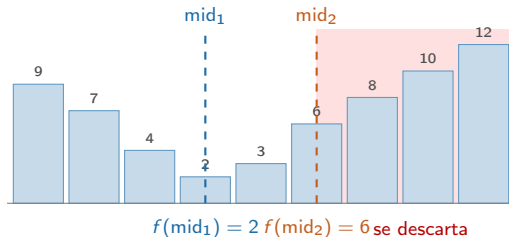
Ejercicio 13 - Qué tercio se descarta



Ejercicio 13 - Qué tercio se descarta



Ejercicio 13 - Qué tercio se descarta



Como $f(mid_1) < f(mid_2)$ y f es unimodal, **no puede** haber un valle a la derecha de mid_2 : para llegar ahí f ya tendría que haber empezado a subir.

Ejercicio 13

3 · Etapas D&C

`minDif(A, B)    (dif1 es  $f(\text{mid}_1)$ )`

```
left, right = 0, n - 1
while right - left > 2:
    mid1 = left + (right - left) // 3
    mid2 = right - (right - left) // 3
    dif1 = abs(A[mid1] - B[mid1])
    dif2 = abs(A[mid2] - B[mid2])
    if dif1 > dif2: left = mid1
    else:          right = mid2

return min(abs(A[i] - B[i]) for i in [left..right])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 13

3 · Etapas D&C

`minDif(A, B)    (dif1 es  $f(\text{mid}_1)$ )`

```
left, right = 0, n - 1
while right - left > 2:
    mid1 = left + (right - left) // 3
    mid2 = right - (right - left) // 3
    dif1 = abs(A[mid1] - B[mid1])
    dif2 = abs(A[mid2] - B[mid2])
    if dif1 > dif2: left = mid1
    else:          right = mid2

return min(abs(A[i] - B[i]) for i in [left..right])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 13

3 · Etapas D&C

`minDif(A, B)` (dif1 es $f(\text{mid}_1)$)

```
left, right = 0, n - 1
while right - left > 2:
    mid1 = left + (right - left) // 3
    mid2 = right - (right - left) // 3
    dif1 = abs(A[mid1] - B[mid1])
    dif2 = abs(A[mid2] - B[mid2])
    if dif1 > dif2: left = mid1
    else:          right = mid2

return min(abs(A[i] - B[i]) for i in [left..right])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 13

3 · Etapas D&C

`minDif(A, B)` (dif1 es $f(\text{mid}_1)$)

```
left, right = 0, n - 1
while right - left > 2:
    mid1 = left + (right - left) // 3
    mid2 = right - (right - left) // 3
    dif1 = abs(A[mid1] - B[mid1])
    dif2 = abs(A[mid2] - B[mid2])
    if dif1 > dif2: left = mid1
    else:          right = mid2

return min(abs(A[i] - B[i]) for i in [left..right])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

- Supongamos que no: que $i^* > \text{mid}_2$.

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

- Supongamos que no: que $i^* > \text{mid}_2$.
- Como f es unimodal y su mínimo está a la derecha de mid_2 , f viene decreciendo en todo $[\text{mid}_1, \text{mid}_2]$.

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

- Supongamos que no: que $i^* > \text{mid}_2$.
- Como f es unimodal y su mínimo está a la derecha de mid_2 , f viene decreciendo en todo $[\text{mid}_1, \text{mid}_2]$.
- En particular $f(\text{mid}_1) > f(\text{mid}_2)$, porque $\text{mid}_1 < \text{mid}_2$.

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

- Supongamos que no: que $i^* > \text{mid}_2$.
- Como f es unimodal y su mínimo está a la derecha de mid_2 , f viene decreciendo en todo $[\text{mid}_1, \text{mid}_2]$.
- En particular $f(\text{mid}_1) > f(\text{mid}_2)$, porque $\text{mid}_1 < \text{mid}_2$.
- Pero habíamos asumido $f(\text{mid}_1) \leq f(\text{mid}_2)$. **Absurdo.**

Ejercicio 13

4 · Correctitud

¿Por qué puedo tirar un tercio entero?

Sea i^* la posición donde f alcanza su mínimo. Si $f(\text{mid}_1) \leq f(\text{mid}_2)$, afirmamos que $i^* \leq \text{mid}_2$, o sea que el último tercio no tiene la respuesta.

- Supongamos que no: que $i^* > \text{mid}_2$.
- Como f es unimodal y su mínimo está a la derecha de mid_2 , f viene decreciendo en todo $[\text{mid}_1, \text{mid}_2]$.
- En particular $f(\text{mid}_1) > f(\text{mid}_2)$, porque $\text{mid}_1 < \text{mid}_2$.
- Pero habíamos asumido $f(\text{mid}_1) \leq f(\text{mid}_2)$. **Absurdo.**

El caso $f(\text{mid}_1) > f(\text{mid}_2)$ es simétrico.

Ejercicio 13

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq 2 \\ T\left(\frac{2n}{3}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

Ejercicio 13

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq 2 \\ T\left(\frac{2n}{3}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

- $a = 1, \quad b = 3/2, \quad f(n) = \Theta(1)$
- $q = \log_b a = \log_{3/2} 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Ejercicio 13

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq 2 \\ T\left(\frac{2n}{3}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

- $a = 1, \quad b = 3/2, \quad f(n) = \Theta(1)$
- $q = \log_b a = \log_{3/2} 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

Ejercicio 13

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n \leq 2 \\ T\left(\frac{2n}{3}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

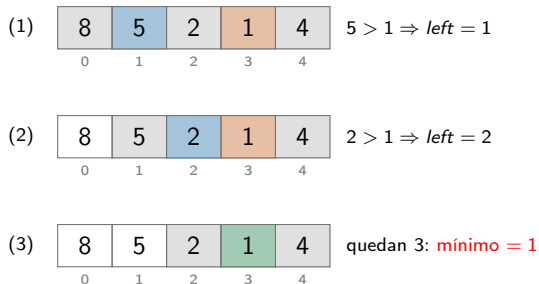
- $a = 1, \quad b = 3/2, \quad f(n) = \Theta(1)$
- $q = \log_b a = \log_{3/2} 1 = 0$, y entonces $f(n) = \Theta(n^q)$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

$T(n) = \Theta(\log n)$ (la base $3/2$ en vez de 2 sólo cambia una constante)

Ejercicio 13 - Ejemplo de ejecución

$A = [1, 2, 3, 4, 5]$, $B = [9, 7, 5, 3, 1]$, $\text{diferencias} = [8, 5, 2, 1, 4]$



En gris la ventana; en azul mid_1 y en naranja mid_2 .

Ejercicio 13 - Para comparar

Algoritmo	Complejidad	Idea
Fuerza bruta	$O(n)$	Calcular las n diferencias
Búsqueda ternaria	$O(\log n)$	Tirar un tercio por paso

Ejercicio 13 - Para comparar

Algoritmo	Complejidad	Idea
Fuerza bruta	$O(n)$	Calcular las n diferencias
Búsqueda ternaria	$O(\log n)$	Tirar un tercio por paso

El patrón que se repite

Igual que en la montaña: la **unimodalidad** es lo que permite descartar sin mirar.

Bonus

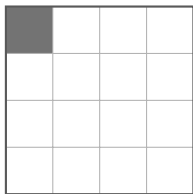


Ejercicio 14 de la guía (*L-Tetris*)

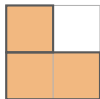
Ejercicio 14 (*L-Tetris*)

Problema

Un tablero de $n \times n$ con n potencia de 2, con **una** posición ya ocupada. Rellenar todas las demás con fichas en forma de L, de 3 posiciones cada una.



entrada



la ficha

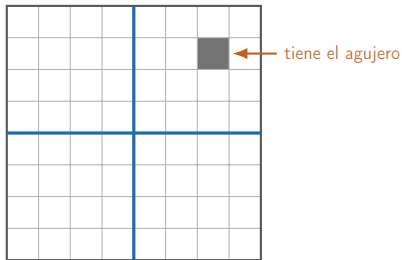


una salida posible

Ejercicio 14

1 · Análisis

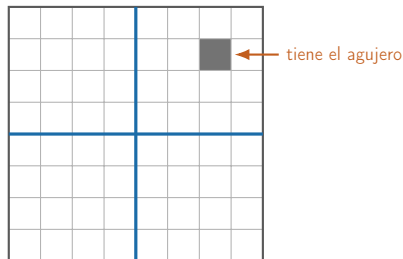
Partimos el tablero en **cuatro** cuadrantes de $\frac{n}{2} \times \frac{n}{2}$:



Ejercicio 14

1 · Análisis

Partimos el tablero en **cuatro** cuadrantes de $\frac{n}{2} \times \frac{n}{2}$:



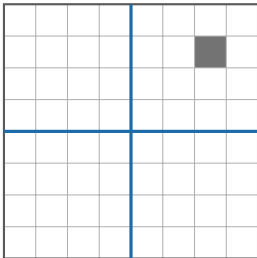
El problema

Sólo **uno** de los cuatro cuadrantes tiene una celda ocupada. Los otros tres están vacíos, así que *todavía no son* el mismo problema.

Ejercicio 14

2 · Algoritmo

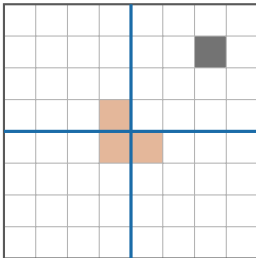
La idea: poner una ficha en el centro, tapando una celda de cada uno de los tres cuadrantes que no tienen el agujero.



Ejercicio 14

2 · Algoritmo

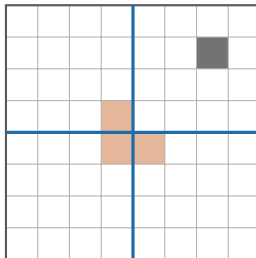
La idea: poner una ficha en el centro, tapando una celda de cada uno de los tres cuadrantes que no tienen el agujero.



Ejercicio 14

2 · Algoritmo

La idea: poner una ficha en el centro, tapando una celda de cada uno de los tres cuadrantes que no tienen el agujero.



Ahora los cuatro cuadrantes tienen exactamente una celda ocupada: son cuatro instancias del mismo problema, de lado $n/2$.

Ejercicio 14

3 · Etapas D&C

lTetris(B, esq, n, ag)

```
# esq = esquina inferior izq., n = lado, ag = celda ocupada
(x, y) = esq
# caso base: las 3 celdas que sobran ya forman una L
if n == 2:
    ponerFicha(B, las otras 3 celdas del cuadrado); return
m = n / 2
E = [(x,y), (x+m,y), (x,y+m), (x+m,y+m)] # esquinas de los cuadrantes
C = [(x+m-1,y+m-1), (x+m,y+m-1), (x+m-1,y+m), (x+m,y+m)] # centro
q = el k tal que ag cae en el cuadrante E[k] # el del agujero
# la ficha del centro tapa una celda de los otros tres
ponerFicha(B, C sin C[q])
# cada cuadrante queda con una sola celda ocupada
for k in 0..3:
    lTetris(B, E[k], m, ag if k == q else C[k])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 14

3 · Etapas D&C

lTetris(B, esq, n, ag)

```
# esq = esquina inferior izq., n = lado, ag = celda ocupada
(x, y) = esq
# caso base: las 3 celdas que sobran ya forman una L
if n == 2:
    ponerFicha(B, las otras 3 celdas del cuadrado); return
m = n / 2
E = [(x,y), (x+m,y), (x,y+m), (x+m,y+m)] # esquinas de los cuadrantes
C = [(x+m-1,y+m-1), (x+m,y+m-1), (x+m-1,y+m), (x+m,y+m)] # centro
q = el k tal que ag cae en el cuadrante E[k] # el del agujero
# la ficha del centro tapa una celda de los otros tres
ponerFicha(B, C sin C[q])
# cada cuadrante queda con una sola celda ocupada
for k in 0..3:
    lTetris(B, E[k], m, ag if k == q else C[k])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 14

3 · Etapas D&C

lTetris(B, esq, n, ag)

```
# esq = esquina inferior izq., n = lado, ag = celda ocupada
(x, y) = esq
# caso base: las 3 celdas que sobran ya forman una L
if n == 2:
    ponerFicha(B, las otras 3 celdas del cuadrado); return
m = n / 2
E = [(x,y), (x+m,y), (x,y+m), (x+m,y+m)] # esquinas de los cuadrantes
C = [(x+m-1,y+m-1), (x+m,y+m-1), (x+m-1,y+m), (x+m,y+m)] # centro
q = el k tal que ag cae en el cuadrante E[k] # el del agujero
# la ficha del centro tapa una celda de los otros tres
ponerFicha(B, C sin C[q])
# cada cuadrante queda con una sola celda ocupada
for k in 0..3:
    lTetris(B, E[k], m, ag if k == q else C[k])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 14

3 · Etapas D&C

lTetris(B, esq, n, ag)

```
# esq = esquina inferior izq., n = lado, ag = celda ocupada
(x, y) = esq
# caso base: las 3 celdas que sobran ya forman una L
if n == 2:
    ponerFicha(B, las otras 3 celdas del cuadrado); return
m = n / 2
E = [(x,y), (x+m,y), (x,y+m), (x+m,y+m)] # esquinas de los cuadrantes
C = [(x+m-1,y+m-1), (x+m,y+m-1), (x+m-1,y+m), (x+m,y+m)] # centro
q = el k tal que ag cae en el cuadrante E[k] # el del agujero
# la ficha del centro tapa una celda de los otros tres
ponerFicha(B, C sin C[q])
# cada cuadrante queda con una sola celda ocupada
for k in 0..3:
    lTetris(B, E[k], m, ag if k == q else C[k])
```

■ Dividir ■ Conquistar ■ Combinar

Ejercicio 14

4 · Correctitud

Invariante

Cada llamada recibe un tablero de lado potencia de 2 con **exactamente una** celda ocupada.

Ejercicio 14

4 · Correctitud

Invariante

Cada llamada recibe un tablero de lado potencia de 2 con **exactamente una** celda ocupada.

- **Caso base** ($n = 2$): quedan 3 celdas libres y forman siempre una L, sea cual sea la ocupada.

Ejercicio 14

4 · Correctitud

Invariante

Cada llamada recibe un tablero de lado potencia de 2 con **exactamente una** celda ocupada.

- **Caso base** ($n = 2$): quedan 3 celdas libres y forman siempre una L, sea cual sea la ocupada.
- **Paso inductivo**: la ficha central deja una celda ocupada en cada uno de los tres cuadrantes sin agujero, y el cuarto ya tenía la suya.

Ejercicio 14

4 · Correctitud

Invariante

Cada llamada recibe un tablero de lado potencia de 2 con **exactamente una** celda ocupada.

- **Caso base** ($n = 2$): quedan 3 celdas libres y forman siempre una L, sea cual sea la ocupada.
- **Paso inductivo**: la ficha central deja una celda ocupada en cada uno de los tres cuadrantes sin agujero, y el cuarto ya tenía la suya.
- Las cuatro llamadas cumplen el invariante $\Rightarrow$ por hipótesis inductiva rellenan su cuadrante.

Ejercicio 14

4 · Correctitud

Invariante

Cada llamada recibe un tablero de lado potencia de 2 con **exactamente una** celda ocupada.

- **Caso base** ($n = 2$): quedan 3 celdas libres y forman siempre una L, sea cual sea la ocupada.
- **Paso inductivo**: la ficha central deja una celda ocupada en cada uno de los tres cuadrantes sin agujero, y el cuarto ya tenía la suya.
- Las cuatro llamadas cumplen el invariante $\Rightarrow$ por hipótesis inductiva rellenan su cuadrante.

Como los cuadrantes son disjuntos, las fichas nunca se pisan.

Ejercicio 14

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 2 \\ 4 T(\frac{n}{2}) + \Theta(1) & \text{si } n > 2 \end{cases}$$

Ejercicio 14

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 2 \\ 4 T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

- $a = 4$ (los cuatro cuadrantes), $b = 2$, $f(n) = \Theta(1)$ (poner una ficha)
- $q = \log_b a = \log_2 4 = 2$, y $f(n) = O(n^{q-\varepsilon})$

Ejercicio 14

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 2 \\ 4 T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

- $a = 4$ (los cuatro cuadrantes), $b = 2$, $f(n) = \Theta(1)$ (poner una ficha)
- $q = \log_b a = \log_2 4 = 2$, y $f(n) = O(n^{q-\varepsilon})$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

Ejercicio 14

5 · Complejidad

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 2 \\ 4 T\left(\frac{n}{2}\right) + \Theta(1) & \text{si } n > 2 \end{cases}$$

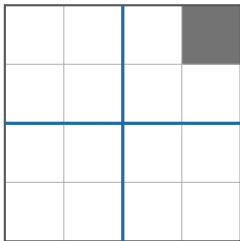
- $a = 4$ (los cuatro cuadrantes), $b = 2$, $f(n) = \Theta(1)$ (poner una ficha)
- $q = \log_b a = \log_2 4 = 2$, y $f(n) = O(n^{q-\varepsilon})$

Caso	Hipótesis sobre $f(n)$	Resultado
1	$f(n) = O(n^{q-\varepsilon})$	$T(n) = \Theta(n^q)$
2	$f(n) = \Theta(n^q \log^r n)$, con $r \geq 0$	$T(n) = \Theta(n^q \log^{r+1} n)$
3	$f(n) = \Omega(n^{q+\varepsilon})$ y $a f(n/b) \leq c f(n)$	$T(n) = \Theta(f(n))$

$$T(n) = \Theta(n^2)$$

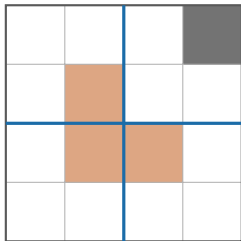
y n^2 es la cantidad de celdas, así que es óptimo

Ejercicio 14 - Ejemplo



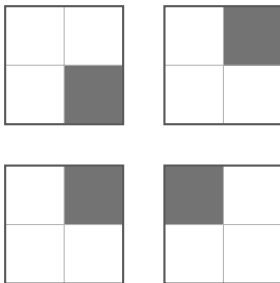
Tenemos el tablero con su única celda ocupada.

Ejercicio 14 - Ejemplo



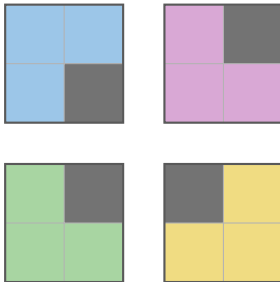
Ponemos la ficha del centro, tapando una celda de cada uno de los otros tres cuadrantes.

Ejercicio 14 - Ejemplo



Ahora esas celdas cuentan como **ocupadas**: los cuatro cuadrantes son la **misma instancia** de 2×2 con una celda ocupada.

Ejercicio 14 - Ejemplo



Cada cuadrante se resuelve solo, con la única ficha que entra.

¿Preguntas?

